
WHITEPAPER

ENVIRONMENT DESIGN FOR REINFORCEMENT LEARNING: A PRACTICAL GUIDE AND OVERVIEW

Thomas Wolgast 

Digitalized Energy Systems
Carl von Ossietzky Universität Oldenburg
Ammerländer Heerstraße 114-118, 26129 Oldenburg,
thomas.wolgast@uni-oldenburg.de

ABSTRACT

The design of Reinforcement Learning (RL) environments has a strong impact on RL training performance and generality of results. While most researchers focus on the agent perspective of RL, the main hindrance to applying RL to real-world problems is the environment design process, not the lack of capable algorithms. That is especially true for RL practitioners, who have only high-level RL knowledge but are experts in their respective domains. Therefore, this work provides a concise guide for practitioners from applied science and industry on how to formulate real-world problems as RL environments to maximize training performance and transferability to the real-world use case. That is done by providing a first definition of RL environment design, followed by guidelines on what to consider when creating RL environments, categorized by their main components: State and observation space, action space, reward function, training data, and episode definition.

Keywords Environment Design · Reward Shaping · Reward Engineering · Reward Design · Action Shaping · Markov Property · Hyperparameter-Optimization · Action Shaping

1 Introduction

The Reinforcement Learning (RL) framework consists of the learning agent as problem solver and the environment as problem representation. Basic RL research focuses almost completely on the agent side, while the problem formulation as RL environment often remains neglected [19]. However, practitioners who attempt to solve real problems with RL will spend most of their time and effort translating their problem to an RL environment. In contrast, for the RL algorithm, most practitioners can simply use open-source implementations of standard algorithms. In conclusion, most research efforts do not really tackle the main hindrance of applying RL to real-world problems, which is environment design. Various studies demonstrated that environment design decisions drastically impact training performance, speed, and generality of results [11, 13, 17, 18, 19, 31, 34, 35]. If done wrong, it can even result in an impossible learning task, which is especially problematic if the solvability of the problem is unknown beforehand [11].

New RL environments are mainly created and published by RL researchers to serve as benchmarks and to evaluate novel RL algorithms. However, such benchmarks cannot serve as guides on how to design good RL environments because they are designed to be difficult while being solvable, usually representing a yet unsolved problem class. In practice, however, we want the opposite, to design environments that support the training performance. In other words, we want to find the easiest-to-solve problem representation. Hence, I will define RL environment design for real-world problems as follows:

Reinforcement Learning Environment Design is the manual or automated process of constructing an RL environment representation of a given problem, aiming for the optimal balance between minimizing computational resources, reducing engineering effort, and maximizing agent performance regarding task-specific metrics.

The goal and contribution of this work is to provide a very first guide on aspects to consider during environment design. This guide’s target groups are practitioners from industry and researchers from other domains outside basic RL research who want to solve domain-specific problems with RL. Members of both groups are usually experts in some domain but have limited RL knowledge, which is why this guide provides a high-level entry point. For expert RL researchers, this work can serve as the first high-level literature survey on environment design since no other survey exists yet.

This guide cannot provide definite answers on what design decisions to take, because the actual optimal environment design is usually highly problem-specific. However, it will provide a guide and recommendations on what to consider during the environment design process, like typical mistakes or tricks. That is done by following the *Gymnasium*¹ API [25], as it is the quasi-standard of implementing RL environments. To start, imagine the situation of a practitioner who needs to convert a given problem to an RL environment and wants to use out-of-the-box standard open-source RL algorithms. Since we assume the usage of standard algorithms, I will point to advanced topics only in special cases without diving deeper.

This work is structured as follows. After an introduction on why I wrote this Whitepaper, Section 3 shortly presents *Gymnasium* and its API. Section 4 presents typical design decisions to make and provides recommendations on what to consider. It is the core of this work and is divided into guidelines on the observation space, the action space, the reward function, the data distribution, the episode definition, and other recommendations. Sections 5 and 6 conclude the work with a conclusion and some final remarks.

2 Why I Wrote This Paper

This work cumulated from my personal experiences in applied RL when implementing environments for power system optimization [30, 31, 32]. In the literature, I noticed a wide variety of environment design decisions to represent the same underlying problem [31], however, without discussion or experimentation on why design A is better than design B. I also realized that my own understanding of the optimal environment implementation was merely intuition without scientific grounding. I suspect the same might be true for many other researchers in applied RL. Finally, when supervising students, I observed that they seemed to struggle more with implementing a new environment than with the RL algorithm. With this Whitepaper, I hope to emphasize the importance of RL environment design, provide a first definition, and give some guidance on what to consider in the process to make practical problems solvable with RL. As many of the points discussed here cannot be supported by literature because so little literature exists on them, this paper was written mainly to be helpful.

3 The *Gymnasium* Environment API Standard

Before discussing specific guidelines, I will first quickly introduce *Gymnasium* [25], which is a fork of OpenAI’s outdated *Gym* library. *Gymnasium* is relevant for two reasons: First, the *Gymnasium* API inherently represents most relevant environment design decisions that we will discuss later. Therefore, I will often refer to specific *Gymnasium* methods later on. Second, *Gymnasium* is the quasi-standard for RL environments, which is supported by most open-source implementations of standard RL algorithms. Since a newly created environment should always be tested with provably working RL algorithms, we should use *Gymnasium*. This way, we know that a failure to learn is due to the environment and not the agent, which will simplify debugging.

The *Gymnasium* API contains two core methods that the agent interacts with: *reset* and *step*. The *reset* method initializes an RL episode and provides the agent with an observation array that represents the initial environment state. This way, *reset* starts the RL interaction loop. Based on the observation, the agent chooses an action and interacts with the environment via the *step* method, which takes the *action* as an input and returns an (*observation*, *reward*, *terminated*, *truncated*, *info*) tuple as feedback. The new *observation* represents the next state and continues the agent-environment interaction loop. The scalar *reward* serves as feedback regarding the quality of the previous action in the given state. The boolean *terminated* indicates the natural end of an episode, while the boolean *truncated* indicates whether a new *reset* is required even when the episode is not terminated. Finally, both *reset* and *step* return an *info* dictionary, which provides additional information to the learning agent or can be used for logging. It can also be used to bypass the *Gymnasium* API to some extent, for example, by providing a vector representation of the reward or a cost value to represent constraint violations.

Gymnasium also provides so-called *wrappers*, which enable us to perform generic modifications on *Gymnasium* environments without the need to change the original environment. For example, wrappers can be used for normalization (see 4.1), to discretize actions (see 4.2), or to add time limits to the environment (see 4.5).

¹<https://gymnasium.farama.org/index.html>, last access 2024-10-14.

4 Typical Environment Design Decisions

The following sections will discuss various aspects to consider when implementing an RL environment. Every RL problem is different and has domain-specific challenges. However, on a high level, the design decisions to make are usually the same. We can differentiate between five categories: observation and state space, action space, reward function, data distribution, and episode definition, which will be discussed below.

4.1 Observation and State Space

The observation is a representation of the current environment state and is used by the agent to determine its next action. Hence, the choice of the observation space is an important design decision for training performance [13, 34]. However, first, it is important to clarify how it differs from the equally important state definition.

Understand state vs. observation The difference between state and observation is very important in practice but often neglected. For example, Sutton and Barto treat them as identical until chapter 17 of their famous fundamentals book "Reinforcement Learning: An Introduction" [24]. The state describes "how the environment is" at a given time [24]. Usually, it is assumed that each state s_t fulfills the Markov property, which means that it contains all the information for making any prediction about the environment [24], like the future state s_{t+n} or the value of that state $V(s_t)$. Therefore, it is required for optimal decision-making.

Ideally, the learning agent directly observes the environment's state, which enables it to learn an optimal policy that maps from states to actions. However, in reality, we or the agent do not necessarily know the current state. Instead, the agent receives the observation, which is a representation of the state. And for real-world problems that representation is usually imperfect. We can differentiate three general categories of imperfectness: partial, redundant, and noisy observations. In partial observability, the agent receives only part of the state as observation. An example would be a 2D image of a 3D environment. Redundant observations are observations that do not provide additional information and are not required for optimal decision-making. An example would be providing coordinates in both cartesian and Euler forms. Noisy observations are all kinds of errors in measuring or communicating the state in stochastic environments, for example, a devious measurement device.

Both state and observation need to be precisely defined when defining a RL environment; the state such that it concisely fulfills the Markov property, the observation so that it maximizes learning performance and is transferable to the intended real-world application. For example, all required measurement data need to be available.

Aim for the Markov property The Markov property was introduced as a characteristic of the environment state. However, since we want to enable the agent to learn an optimal policy, it is useful to aim for the Markov property in the whole environment design process. In other words, how should the environment be designed to allow for optimal decision-making? That is especially relevant when defining the observation space. The observations should enable the agent to infer the environment state for optimal decision-making. Often but not always that means we want to provide as much information to the agent as possible. Various ideas to optimally represent the state are scattered over the RL literature. We will discuss some of them in the following.

A common problem of image observations is that all information about movement is lost. A common solution is to provide a sequence of images instead [13, 15], which allows to infer movement from the image differences. While most intuitive for images, we can apply this idea to all kinds of partially observable problems. In general, a sequence of observations enables the agent to better understand the environment dynamics and the underlying states. Adding the previous actions and rewards as additional information can also be useful for the same reason.

Pardo et al. [17] showed that, in environments with a time limit, the remaining time should be added as observation. Only this way can the agent learn time-dependent policies, for example, more defensive behavior at the end of the episode to save a win over time as often seen in competitive games. Similarly, in real-world environments, it is often useful to provide the current time as sine/cosine encodings as observations, e.g. as demonstrated in [32]. That can be done for day-time, week-time, or even seasonal. For example, energy systems and energy markets are often highly time-dependent.

Redundant observations are not as big of a problem as missing observations. However, Kim and Ha [13] showed that they can be harmful in some cases. We should identify and remove such observations. Note that a never-changing observation channel is always redundant and can be removed. However, removing redundant observations is not as important as adding information-rich observations.

Noisy observations can be expected to slow down training. On the other hand, they can make the policy more robust [19]. The best approach from an environment design perspective is to ensure that the observation distribution during training

fits best with the actual distribution after training (compare section 4.4). For example, if the real-world measurement devices are noisy, the training should be done with equally or even noisier observations to ensure transferability to the real-world problem.

Some first approaches aim to automate the observation shaping process. Yang and Nachum [34] compare multiple offline pretraining approaches for learning state representations before online training. Kim and Ha [13] propose a search algorithm to pick the best observation channels. Note that the second approach not only can be used to pick the best observations in a simulated environment but also to determine optimal sensor placement in the real-world equivalent of the environment.

Respect your data structure Usually, observations are provided as 1-dimensional arrays of independent channels. However, by enforcing that data structure, we may lose a lot of information by omitting potential relations. For example, in the case of image observations, we can use an observation matrix of structure [height, width, color] or [height, width] for black-and-white images. That also requires to use the respective neural networks for this data structure, in this case, Convolutional Neural Networks (CNNs) [15]. If our data is sequential, like text, video, or other sequences (see previous section), we should use Transformers [27] or Long Short-Term Memory (LSTM) networks [8]. If the data has a graph structure, we can use Graph Neural Networks (GNNs) [33]. However, here we face a trade-off regarding open-source availability of RL algorithms that support these. While CNNs and LSTMs are often available, GNNs are usually not supported by open-source RL implementations.

Normalize your observations If the magnitude of observations is highly inhomogeneous, the learning process of most Machine Learning (ML) algorithms gets compromised [29]. Therefore, the observation space should be normalized. In theory, the normalization can be performed on the agent side. However, since our goal is to create RL environments that are usable off-the-shelf, observation scaling should be done on the environment side anyway. Note that this does not hinder another normalization process on the agent side, e.g., by learning a normalization layer [26]. Observation scaling is a typical use case of *Gymnasium* wrappers since they are modularly applicable to all kinds of environments.

The first option for observation scaling is min-max scaling, which scales all observations X to the range $[0, 1]$.

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (1)$$

Min-max scaling is suitable if the maximum and minimum possible observations are known. On the other hand, outliers can result in a massive squashing of the observations. As an example, think about household income as an observation. A single high-income household can increase the overall range drastically. In consequence, even average households will be squashed in the bottom few percent of the observation range.

In case of outliers, or if the range of observations is unbounded, a normalization to a mean of zero and a standard deviation of one can be performed:

$$X' = \frac{X - \mu}{\sigma} \quad (2)$$

with mean μ and standard deviation σ . This variant works well if the observations are normally distributed anyway. However, mean and standard deviation are required to perform the scaling, which are often unknown and may change during training [26]. The problem of changing distributions due to the training process is one reason why normalization on the agent side is even better. However, as stated before, this does not refute the benefit of normalization on the environment side.

Finally, as a third measure, outlier observations can be clipped [15] to deal with distributions with a long tail. This measure can be combined with the previous two and can solve some mentioned problems, e.g., massive outliers or unbounded observation spaces. Clipping can also make training more robust if the outlier observations come from simulation failure or measurement errors.

$$X' = \max(\min(X, X'_{\max}), X'_{\min}) \quad (3)$$

All three approaches require some knowledge about the distribution, which is often not available. As a naive solution, the environment can be randomly stepped a few thousand times to get a first estimate of the distribution. This is far from optimal, especially for long episodes, but can work well in practice, especially if the environment is computationally cheap.

4.2 Action Space

Action space shaping is the task of defining the action space such that it maximizes training performance by simplifying the learning task or exploration.² In that sense, the optimal action space representation is very similar to the Markov property in the observation space. We want to provide all degrees of freedom to the agent that are required for optimal performance. At the same time, we want to remove redundant or detrimental actions to speed-up training. The second point is even more important than for observations because useless actions will sabotage the exploration process.

At first, the choice of the action space representation may seem obvious for many problems. However, this is often not the case [11, 18, 20]. For example, Peng and van de Panne [18] identified multiple action representations for learning the optimal torque in locomotive tasks, later reproduced by Schneider et al. [20]. Interestingly, the intuitive approach to directly learn the optimal torque was significantly outperformed by more high-level representations like target joint-angle velocities. Kanervisto et al. [11] confirmed the strong impact of action representation on training performance in computer game environments like Doom [12] and StarCraft II [28]. However, general rules for good action space representations are difficult to obtain since they are problem-specific by nature. Therefore, action space design requires a lot of domain knowledge about the problem to solve.

Ensure that the agent cannot cheat The agent should never be able to perform actions that are impossible or meaningless in the given state. If impossible actions yield good rewards, the agent will learn a policy that maximizes return but is not applicable to the underlying use case.

In the ideal case, we define the action space so that it is impossible for the agent to perform such actions. That is done by intelligently converting the agent action to meaningful actuator setpoints. Imagine, we want to train an agent to control the torque of an engine. It is better to define the action space as $[-1, 1]$ and interpreting the action a as the share of maximum torque $t = a \cdot t_{\max}$ instead of using the action directly ($t = a$) with action space $[-t_{\max}, t_{\max}]$. At first, these two approaches seem to be equivalent, but the first one allows for easy changing of $t_{\max}$, while the second option may result in impossible actions in that case.

If it is impossible to completely prevent impossible actions or if the utilized RL algorithm is not guaranteed to respect the defined action space, we should repair faulty actions before applying them. Often, rounding and clipping can be used. Additionally, a negative reward can be used as a penalty to push the agent to valid actions. We will discuss this deeper in the reward section 4.3.

Continuous vs. discrete actions Two standard types of action spaces exist in RL: discrete and continuous action spaces. Other action types are possible as well, but we will focus on these two since most RL algorithms only support discrete and/or continuous actions. One of the most common errors is not to respect the natural action space of the problem at hand. Often, that happens by discretizing a normally continuous action, for example, because deep q-learning [15] shall be used. Instead, the algorithm should be chosen based on the characteristics of the environment, not the other way round. Discrete action spaces should only be used when the actions are discrete and independent of each other, for example, pressing button A or B. Continuous action spaces should be used when there is a continuous space to pick from, for example, torque $\in [-1, 1]$. Continuous actions should also be used for discrete but directed actions, for example, torque $\in \{1, 2, 3\}$. Here, the second action is twice as strong as the first one and the third action is even stronger, which is why we should not treat them as independent actions, where that relationship would get lost. Instead, continuous actions should be used because of the directed relationship between the actions. That reduces the dimensionality and helps the agent learn. However, a rounding operator in the environment is required to prevent the agent from selecting impossible actions (see previous paragraph).

A good rule of thumb is that actions in virtual environments are often discrete (like video games), while actions in physical environments representing the real world are often continuous [4]. However, in some cases, continuous actions can be discretized if the exact magnitude does not matter anyway, for example, if we only need to differentiate between high and low.

Finally, it is common practice to use only discrete or continuous actions, not both in the same environment. Again, this allows us to use standard open-source implementations of RL algorithms, which often cannot handle both together. However, that will not always be possible in practice.

²Here I deliberately deviate from Kanervisto et al.'s [11] definition, which assumes the existence of some original action space to be modified. However, when creating a new environment, we have to come up with that original action space by ourselves, which makes our definition here slightly more general.

4.3 Reward Function

The reward function is the core of RL and one of the the more researched environment design topics. In literature, it can be found under "reward engineering", "reward shaping", or "reward design". The following paragraphs will present guidelines on what to consider when defining the reward function.

Be precise and avoid loopholes For the agent to learn a problem-solving policy, the reward function must precisely represent the desired objective to be achieved. Therefore, the maximum possible return should be achieved only by the optimal problem-solving policy. If the reward is badly defined and if it is possible to exploit the reward function without solving the given problem, RL agents are likely to find that exploit. There are countless literature examples of RL agents that successfully maximize rewards without achieving the intended objective, for example, in [16]. It is also important to remember that RL agents do not maximize the reward but the sum of rewards over time. Therefore, the reward function should reflect the underlying long-term goal. Another thing the reward designer should avoid is having a preferred policy in mind and designing the reward to incentivize that exact policy. In that case, the RL will be limited in finding new and superior policies. For example, Alpha Zero [22] would not have surpassed chess grandmasters if rewards were given for moves that were considered good at the time. In such scenarios, RL degrades to inefficient supervised learning. Instead, positive rewards were given only for achieving the actual goal, which was winning [22].

Dense rewards and reward shaping The natural choice of a reward function might be very sparse. For example, in a chess game, providing rewards only at the end of the game will slow down training because the connection between the moves (the actions) and the resulting reward is highly delayed. This is called the credit-assignment problem [24]. To support learning, the reward signal should be as dense as possible [23]. In the ideal case, the agent receives a non-constant reward after every action, and better actions yield better rewards. However, denser rewards also have a higher risk of loopholes (see previous section). Hence, the goal should be to create a denser reward without changing the resulting optimal policy [16]. The procedure to transform a sparse reward into a dense one is called reward shaping. Ng et al. [16] show how potential-based rewards can improve learning speed without altering the resulting optimal policy.

Normalize your reward Very similar to the observations, the reward should be normalized. For example, extreme rewards may result in drastic policy updates during training, which jeopardizes training stability. Henderson et al. [7] found a significant influence of reward scale on training performance. Some RL algorithms like SAC [5] even have explicit countermeasures against badly scaled reward functions. All previous points from the observation scaling section 4.1 apply here as well. However, clipping is far more prevalent for the reward, for example, in Mnih's groundbreaking work [15], where reward clipping was introduced to standardize the reward scale of the Atari game environments. However, note that clipping may change the nature of the problem and the resulting optimal policy [15, 26].

How (not) to consider constraints In some cases, we want to consider constraints within our RL environment. For example, the agent may not be allowed to perform action A in state B, or some state variable is constrained by equality or inequality constraints. Especially the second case is often non-obvious to solve. The naive approach is to use penalty terms in the reward function. However, that results in a series of problems. First of all, constraints must often be considered in every single step, while the RL objective is to maximize reward over all steps. Therefore, an RL agent might accept violating the short-term constraints if the expected future reward is sufficiently high. The natural countermeasure to solve that is by defining an extremely strong penalty. However, that will result in massive outliers and negatively influence the overall reward magnitude (see previous section). Additionally, a big constant penalty is very sparse since it does not provide any direction to move to. Preferable would be a penalty that increases with stronger violations (if applicable) to achieve a denser reward and support training. However, that again will enable the agent to ignore smaller violations.

In conclusion, manually created penalty terms should only be used for simple constraints and/or if smaller violations are acceptable. For more complex constraints, either automatic approaches should be used, e.g. [6], or Safe RL algorithms [36]. Safe RL explicitly considers the constraints in the learning process, and some algorithms can even perform safe exploration during training. However, Safe RL is not directly compatible with the *Gymnasium* API because it requires a *cost* function as a representation of constraint violations.³

³Although strictly speaking, Safe RL is not possible with the *Gymnasium* API, the *info* dictionary can be used to communicate the cost function. There are also specialized Safe RL environment frameworks build upon *Gymnasium* like *Safety Gymnasium* [10].

4.4 Training Data Distribution

The importance of the training data distribution is highly neglected in RL. It is also more complicated compared to other ML methods because the agent dynamically creates its dataset by interacting with its environment.

In RL, the training data is a trajectory of (observation, action, reward)-tuples after an initial observation X_0 . The agent usually can control these trajectories to some extent but not fully. The agent controls its own actions and, therefore, the next state and reward to some extent. However, both are also influenced by stochastic effects, which are out of control. Further, often, some parts of the state cannot be controlled at all. For example, a robot can control its own actions and its location within the environment, but it has no control over outside factors like the terrain or human intervention. From an environment design perspective, the initial state distribution, sampled in the *Gymnasium reset* method, is especially important because it is fully controllable by the environment designer. Reda et al. [19] showed a significant influence of the initial state distribution on final performance.

Use a test dataset Zhang et al. [35] showed that, although often neglected, overfitting is a problem in RL. If the training data is not diverse enough, the agent will simply memorize the best actions instead of learning a strategy that generalizes to unknown observations. They propose using separate test and training datasets in RL, as is common practice in other ML fields. Applied to environment design, this means we should start by defining a test dataset that does not get sampled in the *reset*. That test data should be chosen based on the expected situations where the trained agent must perform well after training. For example, if we train an agent to control a power system, we need real or at least realistic power system states. If the agent interacts with customers, we need realistic test customers and defined test scenarios. Note that the second example is a dataset only in the abstract sense. Ideally, the test dataset also consists of optimal actions or naive baseline actions to evaluate the agent's performance. To have both, optimal and naive actions, is ideal for performance evaluation during and after training.

Choose the training dataset accordingly After defining the test data, we can choose the training data distribution by picking the data that will maximize performance on the test data. The ideal scenario is to use the same data distribution or a large enough subset of the same dataset. However, that is not always possible in real-world problems because the data distribution is not known, and available datasets are too limited. In that case, we need to define the training data distribution wide enough so that it encompasses the test dataset. This will decrease training performance but increase test performance, ensuring that the learned policy is transferable to the actual use case. One approach to do this is by adding noise to existing realistic data samples. In general, all kinds of data augmentation methods are applicable here.

4.5 Episode Definition

RL aims to maximize the sum of rewards over the whole episode. Therefore, the definition of the episode and its termination criteria are very important for learning success. First, we have to differentiate between termination and truncation.

Understand termination vs. truncation Termination marks the end of the episode and of the optimization time frame. This termination signal is used by many RL algorithms to learn that the future expected sum of rewards will be zero, which simplifies learning drastically. After termination, the *reset* method of *Gymnasium* is used to start a new episode.

Truncation requires a reset as well. However, it does not define the end of the episode and does not give us any information about the future expected reward. It is usually triggered by some time or step limit in the environment to prevent infinitely long trajectories. It can also be used to increase the variance of training data or to catch unintended failures in the environment, which require a reset. Confusing termination and truncation is a common mistake in applying RL [17]. Using the truncation signal for setting expected future rewards to zero can significantly deteriorate training success [17] because the agent will over- or underestimate future rewards.

Mix up the data One use case of truncation is to make sure that the agent sees more variant data. For example, imagine training an agent on time-series data that spans multiple months or years on an infinite-horizon task. At the beginning of training, the agent will mainly see winter data from January, February, etc. Later, the agent will have learned to perform well on winter data but will see only summer data. This way, the environment will almost look non-stationary to the agent (compare 4.6), and especially on-policy RL algorithms will suffer from this. One solution to this is to split the time-series data into smaller chunks and sample randomly from these chunks during *reset* [32]. This way, the agent will see a wider variety of data points even early in the training. Note that the agent should still be able to perform well on the long data series (test dataset). Therefore, we need to use truncation here. Otherwise, the agent will learn to maximize the sum of rewards over the chunk, not over the whole sequence [17]. In these shorter chunks, it

is also very important to have a high variance of initial environment states. Otherwise, too-short chunks will limit the agent’s ability to fully explore the environment [17].

Use 1-step environments if possible In some special cases, it is possible to define a 1-step environment where the episode length is always one, for example, when using RL to approximate solutions of optimization problems. If a 1-step environment is possible and makes sense, it should be preferred over an n-step definition because it simplifies training a lot [31]. That is for two reasons: First, the agent does not influence the state distribution anymore, which makes the distribution stationary and simplifies the exploration-exploitation trade-off. Second, learning a value function becomes a supervised learning problem because reward and return are identical. It also simplifies learning environment models and makes auxiliary tools like target networks unnecessary. Finally, note that our problem is now reduced to a contextual bandit [2], a simpler variant of the RL problem, which allows us to use specialized bandit algorithms.

4.6 Other Recommendations

After the previous section discussed various typical environment design decisions, the following section will discuss some other important things to consider in RL environment design.

Perform environment design systematically As demonstrated by the variety of design decisions discussed before, the optimal environment design is often non-obvious. It is important not to settle with the first idea but to approach environment design systematically. The straightforward approach is to implement multiple variants and compare the resulting performance while keeping all other design decisions fixed, for example, as used in [11, 13, 18, 19, 20, 31]. This is great for a few potential variants and for understanding the pros and cons of specific design decisions. However, it fails for large or continuous design spaces and also neglects potential interdependencies [18].

For large and continuous search spaces where interdependencies can be expected, a more systematic and performance-driven approach can be found in Hyperparameter (HP)-optimization [3]. Normally, HP-optimization is used to determine the best HPs for ML algorithms, like learning rate or batch size. However, if we consider the design decisions of an underspecified RL environment⁴ as HPs, we can apply all methods from HP-optimization directly to environment design, for example, methods like random search, bayesian optimization, or iterated racing [3]. For example, when we are unsure if reward function A or B should be used, we can define this as a discrete HP. Or, when we add noise to the data used, we can define the noise range as a continuous HP to optimize the state distribution. In general, we can and should adopt lots of ideas for rigorous experimentation from HP-optimization literature, like nested resampling to prevent overfitting in case of limited data. Bischl et al. [3] provide a great overview on HP-optimization.

At first glance, applying HP-optimization to environment design seems computationally expensive because large numbers of training runs are required. However, the environment design HP-optimization can be done concurrently with the HP-optimization of the RL algorithm, which makes it computationally cheaper. Further, additional performance improvements are possible because RL algorithm and environment get co-optimized.

Use task-specific metrics for environment design Another thing to consider when evaluating and comparing different environment designs is the evaluation metric. It is important to remember that we want to solve real-world problems, not maximize rewards. Since the reward is only a representation of our objective and also an open design decision, it is not suited as an evaluation metric [23]. Instead, we should use task-specific metrics to evaluate the performance, for example, the number of points scored in a game or the number of solved problem instances. Additionally, we can consider training speed and stability as optimization metrics for environment design.

Use domain knowledge to simplify the problem RL environments represent a specific problem in a given domain. To build environments that support learning, we should use as much domain knowledge as possible without restricting the agent too much. In many cases, domain experts will know beforehand that some observations are redundant or what the optimal action in a given state is. In that case, letting the agent learn that knowledge is a waste of training time. Instead, we should design the environment to incorporate this domain knowledge, for example, by removing redundant observations or automatically applying known optimal actions. The RL agent should be seen as a tool to discover unknown strategies instead of reproducing known knowledge. However, it is worth noting that such domain knowledge should only be incorporated if it does not restrict the agent in its exploration of policies. For example, optimal actions should only be applied if they are provably optimal. Redundant observations should only be removed if they do not improve performance. Again, the whole process should be performed systematically with proper experimentation.

⁴The approach of using underspecified environments is very similar to *configurable environments* by Metelli [14], where some environment design decisions can be learned by the RL agent. However, since we focus on standard RL algorithms, this is out-of-scope here.

Keep the environment stationary The concept of stationarity was already mentioned earlier in section 4.5. Stationarity means that the problem to solve stays fixed over the course of the training. This refers especially to the state transition function and the reward function. Stationarity is an important assumption for most standard RL algorithms [24] and should usually be strictly ensured when designing environments. In other words, the environment should represent one immutable problem that does not change during training. However, many real-world problems are not inherently stationary, for example, trading on markets or controlling energy systems. In such cases, advanced methods like continual learning [21] or iterated relearning [9] can be applied, which are out-of-scope here. However, if the underlying use case changes only slowly, it can be possible to update the environment to the new dynamics and retrain the agent from scratch.

If possible, create multiple samples per step The standard case of RL is that a single agent-environment interaction yields a single data sample. However, in some cases, it is possible to create multiple samples from a single interaction, which may improve the sample efficiency of the learning process. One example can be found in goal-based RL literature with Hindsight Experience Replay (HER) [1]. Imagine the following setting: The agent’s goal is to shoot a ball into some net. The position of the net changes with every episode and is observed by the agent. The reward is positive if the shot is successful and zero otherwise. At the beginning of the training, the agent will mostly fail to achieve the goal. The naive approach would be to yield zero rewards. Instead, the idea of HER is to create a second hypothetical sample where the state and observation are changed such that the goal is actually achieved, i.e., by placing the net where the shot went. The reward for that additional sample is positive. This way, HER creates additional samples without environment interaction and ensures that even the unsuccessful initial agent receives positive rewards, which improves training speed. Originally, HER is a replay buffer variant for off-policy RL algorithms. However, the general approach of creating multiple samples from a single environment interaction is applicable to environment design. With slight abuse of the *Gymnasium* API, the *info* dictionary can be used to communicate the additional samples to the agent. However, the approach works only for off-policy RL algorithms.

Document all environment design decisions This work gave an overview of the many required environment design decisions to create RL environments. However, many publications that apply RL to some problem do not provide a full definition of their created RL environment, which is harmful regarding reproducibility and comparability. Instead, when publishing their results, authors should precisely define state and observation space, the action space, the reward function, the utilized data distributions for training and testing, and the episode termination criteria. Considering that the environment design heavily influences the performance and generality of the trained RL agents and also represents underlying assumptions about the given problem, these design decisions are not purely implementation decisions but part of the overall experiment design. Ideally, the environment implementation is published as open-source code to be reusable for other researchers and can serve as a benchmark for future works. In the long term, strict publication of environment design decisions and how they were derived might also provide the RL community with important meta-insights which environment designs are generally preferable over others. While this point is especially relevant for researchers, strict documentation of the decisions made is always a good practice in computer science.

5 Conclusion

This work provided various guidelines to RL practitioners on what to consider when representing real-world problems as RL environments, without diving deep into the underlying theory. To my knowledge, it provides the first definition of environment design. We discussed the *Gymnasium* API and why it is a good default for representing RL problems as environments. Further, we discussed various guidelines on what to consider for RL environment design. Note that this work cannot provide definite rules for optimal environment design since the optimal design is often highly use-case and domain-specific. However, the guidelines can serve as recommendations on what to consider and what to try out during the design process. Table 1 provides a quick summary of all the provided recommendations and guidelines.

For this work, we made some simplifying assumptions regarding the guidelines. We assumed that the *Gymnasium* framework is sufficient to represent the problem and that standard open-source RL implementations are sufficient to solve the problem. However, many real-world problems require to deviate from the standard case. In many cases, more sophisticated solutions are not strictly required but improve solution quality drastically. However, even in these cases, the best approach is often to start with the *Gymnasium* API and open-source RL implementations to achieve a quick minimal example, which can serve as a baseline for improvements later on.

Table 1: Overview of all presented environment design guidelines.

Category	Guideline
Observation and State Space	Understand state vs. observation Aim for the Markov property Respect your data structure Normalize your observations
Action Space	Ensure that the agent cannot cheat Continuous vs. discrete actions
Reward Function	Be precise and avoid loopholes Dense rewards and reward shaping Normalize your reward How (not) to consider constraints
Training Data Distribution	Use a test dataset Choose the training dataset accordingly
Episode Definition	Understand termination vs. truncation Mix up the data Use 1-step environments if possible
Other Recommendations	Perform environment design systematically Use task-specific metrics for environment design Use domain knowledge to simplify the problem Keep the environment stationary If possible, create multiple samples per step Document all environment design decisions

6 Disclaimer and Final Remarks

The literature on RL environment design is very sparse and often badly substantiated. Therefore, this guide is built mostly on experience and cannot serve as a comprehensive literature survey. Rather it should be seen as a practical guide for RL practitioners. However, I hope it also contains some useful points for RL basic researchers, considering that no comprehensive survey paper on environment design exists yet.

This is a work-in-progress paper, which I plan to keep up-to-date with my understanding and knowledge of environment design. Currently, it is not planned for submission at a journal or conference. Feel free to contact me if I forgot important points, for critique of any kind, for pointing out potential errors, for literature recommendations (theoretical or empirical), or for any other feedback. Especially feedback from more experienced RL researchers is highly anticipated and will be considered in future versions of this work. Thank you!

Acknowledgements

Many thanks to Malin Barg, Lorenz Mumm, Stephan Balduin, Arlena Wellßow, and Astrid Nieße for the valuable feedback during internal review.

References

- [1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight Experience Replay. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://papers.nips.cc/paper_files/paper/2017/hash/453fadbd8a1a3af50a9df4df899537b5-Abstract.html.
- [2] Alberto Bietti, Alekh Agarwal, and John Langford. A Contextual Bandit Bake-off. *Journal of Machine Learning Research*, 22(133):1–49, 2021. ISSN 1533-7928. URL <http://jmlr.org/papers/v22/18-863.html>.
- [3] Bernd Bischl, Martin Binder, Michel Lang, Tobias Pielok, Jakob Richter, Stefan Coors, Janek Thomas, Theresa Ullmann, Marc Becker, Anne-Laure Boulesteix, Difan Deng, and Marius Lindauer. Hyperparameter optimization:

- Foundations, algorithms, best practices, and open challenges. *WIREs Data Mining and Knowledge Discovery*, 13(2):e1484, 2023. ISSN 1942-4795. doi: 10.1002/widm.1484.
- [4] Gabriel Dulac-Arnold, Nir Levine, Daniel J. Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. An empirical investigation of the challenges of real-world reinforcement learning, March 2021. URL <https://arxiv.org/abs/2003.11881>.
- [5] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft Actor-Critic Algorithms and Applications, January 2019. URL <http://arxiv.org/abs/1812.05905>.
- [6] Tairan He, Weiye Zhao, and Changliu Liu. AutoCost: Evolving Intrinsic Cost for Zero-Violation Reinforcement Learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(12):14847–14855, June 2023. ISSN 2374-3468. doi: 10.1609/aaai.v37i12.26734.
- [7] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence*, AAAI’18/IAAI’18/EAAI’18, pages 3207–3214, New Orleans, Louisiana, USA, February 2018. AAAI Press. ISBN 978-1-57735-800-8.
- [8] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, November 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL <https://ieeexplore.ieee.org/abstract/document/6795963>.
- [9] Maximilian Igl, Gregory Farquhar, Jelena Luketina, Wendelin Boehmer, and Shimon Whiteson. Transient Non-stationarity and Generalisation in Deep Reinforcement Learning. In *International Conference on Learning Representations*, October 2020. URL <https://openreview.net/forum?id=Qun8fv4qSby>.
- [10] Jiaming Ji, Borong Zhang, Jiayi Zhou, Xuehai Pan, Weidong Huang, Ruiyang Sun, Yiran Geng, Yifan Zhong, Josef Dai, and Yaodong Yang. Safety gymnasium: A unified safe reinforcement learning benchmark. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 18964–18993. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/3c557a3d6a48cc99444f85e924c66753-Paper-Datasets_and_Benchmarks.pdf.
- [11] Anssi Kanervisto, Christian Scheller, and Ville Hautamäki. Action Space Shaping in Deep Reinforcement Learning. In *2020 IEEE Conference on Games (CoG)*, pages 479–486, August 2020. doi: 10.1109/CoG47356.2020.9231687.
- [12] Michał Kempka, Marek Wydmuch, Grzegorz Runc, Jakub Toczek, and Wojciech Jaśkowski. ViZDoom: A Doom-based AI research platform for visual reinforcement learning. In *2016 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8, September 2016. doi: 10.1109/CIG.2016.7860433.
- [13] Joanne Taery Kim and Sehoon Ha. Observation Space Matters: Benchmark and Optimization Algorithm. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1527–1534, May 2021. URL <https://ieeexplore.ieee.org/abstract/document/9561019>.
- [14] Alberto Maria Metelli. Configurable Environments in Reinforcement Learning: An Overview. In Luigi Piroddi, editor, *Special Topics in Information Technology*, pages 101–113. Springer International Publishing, Cham, 2022. ISBN 978-3-030-85918-3. doi: 10.1007/978-3-030-85918-3_9.
- [15] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing Atari with Deep Reinforcement Learning, December 2013. URL <http://arxiv.org/abs/1312.5602>.
- [16] Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, ICML ’99, pages 278–287, San Francisco, CA, USA, June 1999. Morgan Kaufmann Publishers Inc. ISBN 978-1-55860-612-8.
- [17] Fabio Pardo, Arash Tavakoli, Vitaly Levnik, and Petar Kormushev. Time Limits in Reinforcement Learning. In *Proceedings of the 35th International Conference on Machine Learning*, pages 4045–4054. PMLR, July 2018. URL <https://proceedings.mlr.press/v80/pardo18a.html>.
- [18] Xue Bin Peng and Michiel van de Panne. Learning locomotion skills using DeepRL: Does the choice of action space matter? In *Proceedings of the ACM SIGGRAPH / Eurographics Symposium on Computer Animation*, SCA ’17, pages 1–13, New York, NY, USA, July 2017. Association for Computing Machinery. ISBN 978-1-4503-5091-4. doi: 10.1145/3099564.3099567.

- [19] Daniele Reda, Tianxin Tao, and Michiel van de Panne. Learning to Locomote: Understanding How Environment Design Matters for Deep Reinforcement Learning. In *Proceedings of the 13th ACM SIGGRAPH Conference on Motion, Interaction and Games*, MIG '20, pages 1–10, New York, NY, USA, November 2020. Association for Computing Machinery. ISBN 978-1-4503-8171-0. URL <https://dl.acm.org/doi/10.1145/3424636.3426907>.
- [20] Jan Schneider, Pierre Schumacher, Daniel Häufle, Bernhard Schölkopf, and Dieter Buehler. Investigating the Impact of Action Representations in Policy Gradient Algorithms, September 2023. URL <http://arxiv.org/abs/2309.06921>.
- [21] Khadija Shaheen, Muhammad Abdullah Hanif, Osman Hasan, and Muhammad Shafique. Continual Learning for Real-World Autonomous Systems: Algorithms, Challenges and Frameworks. *Journal of Intelligent & Robotic Systems*, 105(1):9, April 2022. ISSN 1573-0409. doi: 10.1007/s10846-022-01603-6.
- [22] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmashan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419): 1140–1144, December 2018. doi: 10.1126/science.aar6404.
- [23] Henry Sowerby, Zhiyuan Zhou, and Michael L. Littman. Designing Rewards for Fast Learning, May 2022. URL <http://arxiv.org/abs/2205.15400>.
- [24] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning Series. The MIT Press, Cambridge, Massachusetts, second edition edition, 2018. ISBN 978-0-262-03924-6.
- [25] Mark Towers, Jordan K Terry, Ariel Kwiatkowski, John U. Balis, Gianluca Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Arjun KG, Markus Krimmel, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Andrew Jin Shen Tan, and Omar G. Younis. Gymnasium. Zenodo, February 2024. URL <https://zenodo.org/records/10655021>.
- [26] Hado P van Hasselt, Arthur Guez, Arthur Guez, Matteo Hessel, Volodymyr Mnih, and David Silver. Learning values across many orders of magnitude. In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL <https://proceedings.neurips.cc/paper/2016/hash/5227b6aaf294f5f027273aebf16015f2-Abstract.html>.
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- [28] Oriol Vinyals, Timo Ewalds, Sergey Bartunov, Petko Georgiev, Alexander Sasha Vezhnevets, Michelle Yeo, Alireza Makhzani, Heinrich Küttler, John Agapiou, Julian Schrittwieser, John Quan, Stephen Gaffney, Stig Petersen, Karen Simonyan, Tom Schaul, Hado van Hasselt, David Silver, Timothy Lillicrap, Kevin Calderone, Paul Keet, Anthony Brunasso, David Lawrence, Anders Ekermo, Jacob Repp, and Rodney Tsing. StarCraft II: A New Challenge for Reinforcement Learning, August 2017. URL <http://arxiv.org/abs/1708.04782>.
- [29] Simon Wiesler and Hermann Ney. A Convergence Analysis of Log-Linear Training. In *Advances in Neural Information Processing Systems*, volume 24. Curran Associates, Inc., 2011. URL https://papers.nips.cc/paper_files/paper/2011/hash/e836d813fd184325132fca8edcdeb40e-Abstract.html.
- [30] Thomas Wolgast and Astrid Nieße. Approximating Energy Market Clearing and Bidding With Model-Based Reinforcement Learning. *IEEE Access*, 12:145106–145117, 2024. ISSN 2169-3536. doi: 10.1109/ACCESS.2024.3472480.
- [31] Thomas Wolgast and Astrid Nieße. Learning the optimal power flow: Environment design matters. *Energy and AI*, page 100410, August 2024. ISSN 2666-5468. doi: 10.1016/j.egyai.2024.100410.
- [32] Thomas Wolgast, Eric MSP Veith, and Astrid Nieße. Towards reinforcement learning for vulnerability analysis in power-economic systems. *Energy Informatics*, 4(3):21, September 2021. ISSN 2520-8942. doi: 10.1186/s42162-021-00181-5.
- [33] Lingfei Wu, Peng Cui, Jian Pei, Liang Zhao, and Xiaojie Guo. Graph Neural Networks: Foundation, Frontiers and Applications. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, pages 4840–4841, New York, NY, USA, August 2022. Association for Computing Machinery. ISBN 978-1-4503-9385-0. doi: 10.1145/3534678.3542609.
- [34] Mengjiao Yang and Ofir Nachum. Representation Matters: Offline Pretraining for Sequential Decision Making. In *Proceedings of the 38th International Conference on Machine Learning*, pages 11784–11794. PMLR, July 2021. URL <https://proceedings.mlr.press/v139/yang21h.html>.

- [35] Chiyuan Zhang, Oriol Vinyals, Remi Munos, and Samy Bengio. A Study on Overfitting in Deep Reinforcement Learning, April 2018. URL <http://arxiv.org/abs/1804.06893>.
- [36] Weiye Zhao, Tairan He, Rui Chen, Tianhao Wei, and Changliu Liu. State-wise Safe Reinforcement Learning: A Survey. In *Thirty-Second International Joint Conference on Artificial Intelligence*, volume 6, pages 6814–6822, August 2023. doi: 10.24963/ijcai.2023/763.